

dut_env.vr

File location: \$VCS_HOME/doc/examples/nativetestbench/openvera/rvm//host_if/env/dut_env.vr

```
/*
 *
 * Author:      Fabian Delguste
 * File:        $RCSfile: dut_env.vr,v $
 * Revision:    $Revision: 1.3 $
 * Date:        $Date: 2003/12/22 15:58:17 $
 *
 ****
 *
 * Basic Transaction Verification Module aimed at
 * creating transactions based on Intel management Interface
 *
 ****
 */

#include <rvm_std_lib.vrh>
#include "intel_mgmt.vrh"
#include "sig_dut.if.vri"
#include "scoreboard.vrh"
#include "coverage.vrh"
#include "bindings.vri"

// Integrate scoreboard in the monitor callback structure
class intel_mgmt_mon_callbacks_scoreboarding extends intel_mgmt_mon_callbacks {
    local scoreboard sb;
    local coverage_model cm;
    task new(scoreboard sb,
              coverage_model cm) {
        this.sb = sb;
        this.cm = cm;
    }

    virtual task post_mon(intel_mgmt_mon    mon,
                          intel_mgmt_trans tr,
                          var bit          drop) {
        if (mon.log.start_msg(mon.log.NOTE_TYP, mon.log.NORMAL_SEV)) {
            if (mon.log.text("Observed transaction:")) {
                void = mon.log.text();
                tr.display("\t  ");
            }
        }
    }
}
```

```

        }
        mon.log.end_msg();
    }
    case (tr.kind) {
        intel_mgmt_trans::READ: {
            this.sb.expect_register(tr.addr, tr.data);
        }
        intel_mgmt_trans::WRITE: {
            this.sb.write_register(tr.addr, tr.data);
        }
    }
    this.cm.cover(tr);

    // Make sure transactions are sunk
    drop = 1;
}
}

class env extends rvm_env {
    test_cfg          cfg;    // Test config
    intel_mgmt_gen     gen;    // Transaction generator
    intel_mgmt_master  mst;    // Transactor
    intel_mgmt_mon     mon;    // Monitor
    scoreboard         sb;
    coverage_model     cm;
    event end_simulation;

    task new() {
        super.new();
        this.cfg = new;
    }

    virtual task gen_cfg() {
        super.gen_cfg();
        void = this.cfg.randomize();
    }

    virtual task build() {
        super.build();

        this.mst = new("", 0, intel_port_bind, this.cfg.intel_mgmt);
        this.mon = new("", 0, intel_port_bind);
        this.gen = new("", 0, this.mst.in_trans);
        this.sb = new(this.cfg);
    }
}

```

```

    this.cm = new;
    {
        intel_mgmt_mon_callbacks_scoreboarding cb = new(this.sb, cm);
        this.mon.register_callback(cb);
    }
}

virtual task cfg_dut_t() {
    super.cfg_dut_t();
    this.reset_dut_t();
}

virtual task reset_dut_t() {
    intel_port_bind.$rst = 1'b0;
    @5 intel_port_bind.$rst = 1'b1;
}

virtual task start_t() {
    super.start_t();
    gen.stop_after_n_scenarios = this.cfg.run_for;
    gen.start_xactor();
    mst.start_xactor();
    mon.start_xactor();
    // End the simulation ASAP by default.
    trigger(ON, this.end_simulation);
}

virtual task wait_for_end_t() {
    super.wait_for_end_t();
    // Wait for the generator to be done
    void = gen.notify.wait_for_t(gen.XACTOR_STOPPED);
    // Wait for a confirmation of the end of simulation
    sync(ALL, this.end_simulation);
    // Then wait for the master to complete the last transaction
    while (mst.in_trans.level() > 0) {
        void = mst.notify.wait_for_t(mst.TRANS_DONE);
    }
    // Wait some more to make sure the monitor reports the last
    // transaction that just completed
    delay(100);
}

virtual task stop_t() {
    super.stop_t();
    gen.stop_xactor();
}

virtual task report() {

```

```
        super.report();  
    }  
}
```